

Sistema de Monitorização Ambiental para a Universidade de Évora

Pedro Carmo 58946
Rodrigo Barreto 58023

January 2026

1 Introdução

O objetivo deste trabalho é desenvolver um sistema distribuído para monitorizar a temperatura e a humidade de diversas áreas da Universidade de Évora. O sistema deverá receber dados enviados por diferentes tipos de dispositivos IoT utilizando múltiplos protocolos de comunicação (MQTT, gRPC e REST), processá-los, armazená-los e disponibilizar funcionalidades para consulta e gestão.

2 Base de Dados

Para o desenvolvimento deste trabalho foram necessárias 3 tabelas de base de dados, sendo estas

1. dispositivo
2. métricas
3. sala

Schema	Name	Type	Owner
public	dispositivo	table	trabalho2
public	métricas	table	trabalho2
public	sala	table	trabalho2

Figure 1: Conjunto de tabelas

2.1 Tabela - dispositivo

A tabela dispositivo, tal como o nome indica, funciona para armazenar-mos todos os dispositivos, para tal, temos as variáveis:

1. id
2. estado
3. nome
4. tipo
5. sala_id

Table "public.dispositivo"				
Column	Type	Collation	Nullable	Default
id	integer		not null	nextval('dispositivo_id_seq'::regclass)
estado	character varying(255)			
nome	character varying(255)			
tipo	character varying(255)			
sala_id	integer			
Indexes:				
"dispositivo_pkey" PRIMARY KEY, btree (id)				
Foreign-key constraints:				
"fkarcytnvfnlostdkksioxcoh" FOREIGN KEY (sala_id) REFERENCES sala(id)				
referenced by:				
TABLE "métricas" CONSTRAINT "fk37ym01fbscp85lrtknghw33bg" FOREIGN KEY (dispositivo_id) REFERENCES dispositivo(id)				

Figure 2: Tabela dispositivo

A variável "sala_id" na verdade é um objeto sala, uma vez que foi utilizado JPA/Hibernate.

2.2 Tabela - métricas

A tabela métricas, tal como o nome indica, funciona para armazenar-mos todas as métricas tiradas pelos dispositivos em determinada sala, para tal conta-mos com as variaves :

1. id (serial)
2. humidade
3. temperatura
4. time_stamp
5. dispositivo_id
6. sala_id

O mesmo se verifica aqui, tanto as variaveis "dispositivo_id" e "sala_id" sao objetos Sala e Dispositivo, porém o JPA/Hibernate trata da conversão.

Table "public.métricas"				
Column	Type	Collation	Nullable	Default
id	bigint		not null	nextval('"métricas_id_seq"::regclass)
humidade	double precision		not null	
temperatura	double precision		not null	
timestamp	timestamp without time zone			
dispositivo_id	integer			
sala_id	integer			

Indexes:

"métricas_pkey" PRIMARY KEY, btree (id)

Foreign-key constraints:

"fk37ym01fbscp85Lrtknghw33bg" FOREIGN KEY (dispositivo_id) REFERENCES dispositivo(id)

"fkynu3l5etjlrjblvq1et06j56t" FOREIGN KEY (sala_id) REFERENCES sala(id)

Figure 3: Tabela Métricas

2.3 Tabela - sala

A tabela sala, tal como o nome indica, funciona para armazenar-mos todas as salas que o nosso sistema pretende registar métricas, para tal as seguintes variáveis são utilizadas :

1. id (serial)
2. andar
3. departamento
4. edificio
5. numero
6. tipo

Table "public.sala"				
Column	Type	Collation	Nullable	Default
id	integer		not null	nextval('sala_id_seq'::regclass)
andar	integer		not null	
departamento	character varying(255)			
edificio	character varying(255)			
numero	integer		not null	
tipo	character varying(255)			
Indexes:				
"sala_pkey" PRIMARY KEY, btree (id)				
Referenced by:				
TABLE "métricas" CONSTRAINT "fkanu3l5etjlrjblvq1et06j56t" FOREIGN KEY (sala_id) REFERENCES sala(id)				
TABLE "dispositivo" CONSTRAINT "fkarcytncefnlostdkksioxcoh" FOREIGN KEY (sala_id) REFERENCES sala(id)				

Figure 4: Tabela sala

3 Métricas

As métricas enviadas são bastante simples. É randomizado um valor tanto para a humidade como para a temperatura, de seguida a cada iteração (a cada 5 segundos) decide-se se se pretende descer ou subir, e a variação também é randomizada, entre 0 - 1.50.

4 Cliente Admin

Este cliente foi desenvolvido em REST é aqui que se podem consultar as métricas tiradas por:

1. dispositivo
2. departamento
3. piso

4. edifício

Pode se também gerir os dispositivos, isto é

1. listar todos os dispositivos
2. adicionar novo dispositivo
3. atualizar dispositivo existente
4. remover dispositivo
5. visualizar detalhes

E por fim pode também consultar estatísticas do sistema.

5 Clientes IoT

Todos os clientes são inicializados mesmo que não estejam ativos. As métricas apenas são inseridas caso o cliente admin ative dispositivo ,desta forma a base de dados começa a aceitar as métricas.

5.1 Cliente MQTT

Utilizamos como broker o Mosquitto. O nosso cliente publica para um tópico específico de forma a indicar ao servidor exatamente qual é o dispositivo, indicamos qual é o edifício/sala.

O cliente está preparado para enviar métricas a cada 5 seg.

5.2 Cliente REST

Todos os clientes REST são inicializados porém as métricas são ignoradas até que algum dos dispositivos que eles estejam associados seja ativo pelo admin. Tal como no MQTT as métricas são enviadas a cada 5 seg.

Temos uma classe "pre-metrica" que o nosso cliente rest cria um objeto e envia-a.

5.3 Cliente gRPC

No cliente gRPC temos uma mensagem no proto chamada Metric, em que nela possui :

1. temperatura
2. humidade
3. idSala
4. idDispositivo

5. timestamp

E apenas um serviço, que enviamos essa tal métrica e esperamos a resposta de um servidor que é uma String.

Tal como nos outros clientes as métricas são enviadas a cada 5 segundos e são geradas da mesma forma.

5.4 Analise de Performance

Para esta parte do trabalho foram inseridas linhas de código que neste momento estão comentadas, os métodos de avaliação de desempenho foram através de **throughput** e da média da **latência** de receber uma mensagem.

5.4.1 REST

Utilizei dois testes diferentes, um com um apenas cliente a enviar métricas e sem o thread sleep, isto é, sem ser a enviar apenas a cada 5 segundos.

1 CLIENTE

Nº de mensagens recebidas : 1288

Duração do teste : 13 segundos

Throughput : 99

Latência : 5ms

100 CLIENTE

Nº de mensagens recebidas : 10345

Duração do teste : 16 segundos

Throughput : 646

Latência : 155ms

5.4.2 gRPC

Utilizei dois testes diferentes, um com um apenas cliente a enviar métricas e sem o thread sleep, isto é, sem ser a enviar apenas a cada 5 segundos.

1 CLIENTE

Nº de mensagens recebidas : 3771

Duração do teste : 11 segundos

Throughput : 342

Latência : 1ms

100 CLIENTE

Nº de mensagens recebidas : 31142
Duração do teste : 14 segundos
Throughput : 2224
Latência : 6ms

5.4.3 MQTT

Realizamos teste para o MQTT porem com as mesmas bases que fazíamos os outros não iríamos obter resultados justos, isto porquê? porque o broker a partir do momento que recebe uma mensagem e se apercebe que está a receber muitas mais ele em vez de as publicar para o cliente logo decide não o fazer, ele fica a espera até ficar cheio e só depois é que publica, assim espera-se que a latência seja grande e que o throughput seja muito bom. Ainda assim aqui vão os resultados, como habitual um teste foi efetuado com apenas um cliente a enviar métricas e outro com 100 clientes a envia-las.

1 CLIENTE

Nº de mensagens recebidas : 1962
Duração do teste : 17 segundos
Throughput : 115
Latência : 8812ms

100 CLIENTE

Nº de mensagens recebidas : 43964
Duração do teste : 17 segundos
Throughput : 2586
Latência : 6566ms

5.4.4 Conclusão da Análise de Performance

O REST revelou limitações evidentes quando sujeito a cargas elevadas. Embora funcione adequadamente com um único cliente, a latência aumenta de forma acentuada com 100 clientes, e o throughput cresce apenas de forma moderada. Isto confirma que, o REST não é a solução mais eficiente para sistemas que exigem comunicação intensiva e em tempo real.

O gRPC demonstrou o melhor equilíbrio entre desempenho e escalabilidade. A latência manteve-se extremamente baixa mesmo com 100 clientes, e o throughput aumentou de forma expressiva. Este comportamento confirma

que gRPC é particularmente adequado para sistemas distribuídos que exigem elevada eficiência.

O MQTT apresentou um comportamento distinto, influenciado pelo funcionamento interno do broker. Apesar de alcançar um throughput muito elevado, a latência registrada foi bastante superior às restantes tecnologias. Isto deve-se ao mecanismo de buffering do broker, que prioriza a entrega eficiente de grandes volumes de mensagens, sacrificando a rapidez de entrega individual. Assim, o MQTT mostra-se ideal para cenários onde a taxa de envio é elevada e a latência não é crítica, mas menos adequado quando é necessária resposta imediata.

6 Observações sobre o desenvolvimento e eventuais desafios encontrados

Foi um trabalho que gostamos muito de desenvolver, com diferentes protocolos e tecnologias. O principal desafio foi saber dividir o funcionamento dos dispositivos quase como coisas independentes do cliente admin, inicialmente a nossa ideia tinha sido sempre que o cliente admin registava um dispositivo ele iniciava uma thread a corre-lo, porém isso ia contra aquilo que era esperado do trabalho.